Package 'SCP'

October 28, 2026

Type Package

Title Power Evaluation and Study Design for Circadian Biomarker Detection

Version 0.5.1

Date 2026-10-28

Maintainer Thien Quy Pham <qtp1@pitt.edu>

URL https://github.com/quythien/SCP

BugReports https://github.com/quythien/SCP/issues

Description Simulation-based power analysis for circadian transcriptomics studies. Empirical parameter distributions (amplitude, noise, phase, proportion rhythmic) are estimated from a pilot dataset, and power is evaluated for rhythmicity detection and differential-rhythm tests (DR, DP, DM) across a range of study designs. Detection uses a unified cosinor F-test detect_cosinor(K), with K = 1 for the single-harmonic case and K = 2 for the two-harmonic (ultradian) extension. A bootstrap design-grid optimiser is included for design comparison under pilot uncertainty.

Depends R (>= 4.4.0)

Imports ggplot2,
limma,
limorhyde,
minpack.lm,
nloptr,
parallel,
grid,
grDevices,
stats,
graphics,
utils,
Rcpp

LinkingTo Rcpp,
RcppArmadillo

Suggests shiny,
ragg,
shinyscreenshot,
DT,
writexl,

enrichR,
org.Hs.eg.db,
org.Mm.eg.db,
AnnotationDbi,
FMM,
MetaCycle,
rain,
circacompare,
patchwork,
cowplot,
gtable,
readxl,
tidyr,
AWFisher,
DODR,
BiocManager,
knitr,
rmarkdown,
testthat (>= 3.1.7)

Config/testthat/edition 3

License MIT + file LICENSE

Encoding UTF-8

LazyData false

biocViews Software, GeneExpression, TimeCourse, DifferentialExpression, Transcriptomics

RoxygenNote 7.3.3

VignetteBuilder knitr

Contents

SCP-package 4
.buildBioFromBoot 5
.collapseTail 6
.load_pilot_data 7
.prepDiffStratified 7
.resampleSubjectIndices 8
.selectTimePoints 8
add_se_bars 9
adjust.to.2pi 9
alpha.adjust 10
amp.cut 10
bootstrapParams 11
calculate_CI.M 11
calculate_CI_A.phase.Taylor 12
choose.abs.min 12
CircadianAnalysisOptions 13
CircadianBioOptions 14
CircadianBootstrapOptions 16
CircadianDesignOptions 17
circaPowerApproxN80 18
circular_concentration 18

circular_difference . . . 19
circular_mean . . . 19
CP_OneGroup . . . 20
detect_cosinor . . . 20
estCircadianParam . . . 21
estCircadianParam2H . . . 23
estCircadianParamTwoGroup . . . 25
fishers.p . . . 26
forwardStop . . . 26
get_phase . . . 27
launchShiny . . . 27
LR_diff . . . 28
makeAdaptiveRStrata . . . 29
npower . . . 29
one_cosinor_OLS . . . 31
plot.SCPDiffResult . . . 31
plot.SCPRecommendResult . . . 32
plot.SCPSingleResult . . . 32
plotBootstrapDesignGrid . . . 33
plotDiffPower . . . 34
plotSingleCohortPower . . . 35
prepCircadianData . . . 38
print.CircadianAnalysisOptions . . . 39
print.CircadianBioOptions . . . 40
print.CircadianBootstrapOptions . . . 40
print.CircadianDesignOptions . . . 41
print.npower . . . 41
print.SCPDiffResult . . . 42
print.SCPRecommendResult . . . 42
print.SCPSingleResult . . . 43
rayleigh_test_circular . . . 43
recommendDesign . . . 44
runBootstrapDesignGrid . . . 45
runDifferentialPower . . . 47
runSimsSingleCohort . . . 48
sampleTimesFromDist . . . 50
scp_load_pilot . . . 50
scp_pilot_database . . . 52
scp_pilot_search . . . 52
scp_pilots . . . 53
SeqModelSel . . . 54
simCircadianDiffFMM . . . 54
simCircadianSingleCohort2H . . . 55
StopRule . . . 56

SCP-package *SCP: Power Evaluation and Study Design for Circadian Biomarker Detection*

Description

SCP (Simulation-based Circadian Power) is a framework for sample-size planning in circadian omics studies, where limited sample availability, uncontrolled collection times, and the multiple-testing burden of genome-wide analysis routinely constrain the design. It calibrates the analysis to a user's pilot dataset, drawing on the pilot's gene-level distributions of amplitude, noise, and phase together with its time-of-day sampling to capture transcriptome-wide signal heterogeneity. From this calibration it estimates FDR-controlled power for two goals: single-cohort detection of rhythmic biomarkers, and two-group differential analysis of rhythmicity, phase, or mesor. A bootstrap layer quantifies the uncertainty that the finite pilot contributes to each power estimate. Detection supports single-harmonic and two-harmonic cosinor models. It bundles curated circadian transcriptomic pilots spanning many tissues and several species, plus a Shiny application for point-and-click power calculation and biomarker exploration.

GTEx input time

GTEx pilots use normalized release v10 expression with season-adjusted time of death (TOD) in the package and app. Gene acrophase is estimated relative to TOD. Within-tissue TMM normalization and log-CPM calculation follow the normalization step of Talamanca et al. (2023). The builder retains samples with valid TOD, keeps the first gene for each symbol, and selects genes with mean log-CPM greater than 3. Cosinor models are fitted to normalized expression. Use `citation("SCP")` for software and source citations.

Main functions

Pilot database • `scp_pilots`
- `scp_load_pilot`
- `scp_pilot_search`

Calibrate your own pilot • `estCircadianParam`
- `estCircadianParam2H`
- `estCircadianParamTwoGroup`
- `prepCircadianData`

Design and options • `CircadianBioOptions`
- `CircadianDesignOptions`
- `CircadianAnalysisOptions`
- `CircadianBootstrapOptions`

Run power analysis • `runSimsSingleCohort`
- `runDifferentialPower`
- `runBootstrapDesignGrid`
- `npower`
- `circaPowerApproxN80`
- `recommendDesign`

Plot results • `plotSingleCohortPower`
- `plotDiffPower`

- plotBootstrapDesignGrid

Detect rhythmicity • detect_cosinor

Simulate and helpers • simCircadianSingleCohort2H

- makeAdaptiveRStrata

App • launchShiny

Author(s)

Maintainer: Thien Quy Pham <qtp1@pitt.edu> (University of Pittsburgh)

Authors:

- Hao Wang (University of Pittsburgh)
- Marianne L. Seney (University of Pittsburgh)
- Colleen A. McClung (University of Pittsburgh)
- Guan Yu (University of Pittsburgh)
- George C. Tseng <ctseng@pitt.edu> (University of Pittsburgh)

References

Talamanca L, Gobet C, Naef F (2023). Sex-dimorphic and age-dependent organization of 24-hour gene expression rhythms in humans. Science 379:478-483. doi:10.1126/science.add0846.

GTEx Consortium (2020). The GTEx Consortium atlas of genetic regulatory effects across human tissues. Science 369:1318-1330. doi:10.1126/science.aaz1776.

See Also

Useful links:

- https://github.com/quythien/SCP
- Report bugs at https://github.com/quythien/SCP/issues

.buildBioFromBoot *Build CircadianBioOptions from a Bootstrap Parameter Draw*

Description

Constructs a CircadianBioOptions object from a single bootstrap resample of pilot-gene cosinor fits. Differential parameters (prop_DR, prop_DP, phase_diff, etc.) are inherited from bio_diff.opts. Amplitude and sigma are kept jointly sampled to preserve the empirical A-sigma correlation.

Usage

```
.buildBioFromBoot(boot_df, bio_diff.opts)
```

Arguments

boot_df	Data frame. One bootstrap resample of fitCosinorAll() output (columns: M, A, phi, sigma, is_rhythmic).
bio_diff.opts	List or CircadianBioOptions. Source for differential-design parameters (prop_DR, prop_DP, phase_diff, etc.).

Value

A `CircadianBioOptions` object ready for `simCircadianSingleCohort()`.

`.collapseTail` *Collapse High-r Tail Strata into a Single Bin*

Description

Merges all r-strata whose left boundary is at or above `threshold` into a single "r >= threshold" bin. This prevents sparse high-r strata from cluttering stratified differential power plots.

Usage

```
.collapseTail(
  r_breaks,
  strata_labels,
  gene_counts,
  mean_TD,
  se_TD,
  mean_pow,
  se_pow,
  threshold = 5
)
```

Arguments

`r_breaks`	Numeric vector. Stratum breakpoints (length = n_strata + 1).
`strata_labels`	Character vector. Labels for each stratum.
`gene_counts`	Integer vector. Gene counts per stratum per simulation.
`mean_TD`	Numeric vector. Mean true discoveries per stratum.
`se_TD`	Numeric vector. SE of true discoveries per stratum.
`mean_pow`	Numeric vector. Mean power per stratum.
`se_pow`	Numeric vector. SE of power per stratum.
`threshold`	Numeric. r value above which strata are collapsed (default 5).

Value

Named list with collapsed vectors: `strata_labels`, `gene_counts`, `mean_TD`, `se_TD`, `mean_pow`, `se_pow`, and scalar `n_strata`.

.load_pilot_data *Load built-in pilot dataset by name*

Description

Load built-in pilot dataset by name

Usage

```
.load_pilot_data(name)
```

Arguments

name	Dataset name (e.g., "ba11_ba47_younger")

Value

List with lBaselineExpr, lOD, amplitude, phase, prop_rhythmic

.prepDiffStratified *Prepare Differential Power Data for Stratified Plotting*

Description

Converts a raw `runSimsDiff()` result into stratified power and true-discovery arrays indexed by sample size, r-stratum, FDR threshold, and simulation replicate.

Usage

```
.prepDiffStratified(res, ep, fdr_thresholds, r_breaks)
```

Arguments

res	List. Output of `runSimsDiff()` (contains `fdr_DR`, `fdr_DP`, `fdr_DM`, `diff_type`, `effectsize`).
ep	Character. Endpoint: one of `"DR"`, `"DP"`, `"DM"`.
fdr_thresholds	Numeric vector. FDR thresholds to evaluate power at.
r_breaks	Numeric vector. Breakpoints defining r-strata (e.g. `c(0, 0.5, 1, 2, Inf)`).

Value

Named list with arrays `power_arr`, `TD_arr`, `marginal_sim`, and matrix `count_mat`.

.resampleSubjectIndices *Reproducible subject-resample indices for the subject bootstrap*

Description

Draws nboot sets of n_pilot subject indices with replacement (Efron nonparametric bootstrap over subjects). The RNG state is saved and restored so the draws are reproducible from seed without disturbing the caller's random stream.

Usage

```
.resampleSubjectIndices(n_pilot, nboot, seed = 42)
```

Arguments

n_pilot	Number of pilot subjects (columns of the pilot matrix).
nboot	Number of bootstrap draws.
seed	Random seed.

Value

List of length nboot; each element is an integer index vector of length n_pilot.

.selectTimePoints *Select B Evenly-Spaced Time Points from a Design Vector*

Description

Select B Evenly-Spaced Time Points from a Design Vector

Usage

```
.selectTimePoints(design_vector, B)
```

Arguments

design_vector	Numeric vector. Full set of candidate ZT time points.
B	Integer. Number of distinct time bins to select.

Value

Numeric vector of length min(B, length(design_vector)) with evenly-spaced time points drawn from design_vector.

add_se_bars *Add +/-1 SE Error Bars to a Base-R Plot*

Description

Add +/-1 SE Error Bars to a Base-R Plot

Usage

```
add_se_bars(x, y, se, col, bar_width = 0.3)
```

Arguments

x	Numeric vector. x-coordinates of the bar centres.
y	Numeric vector. y-coordinates (point estimates).
se	Numeric vector. Standard errors (bar half-width).
col	Colour passed to `arrows()`.
bar_width	Numeric. Relative width of the bar caps (default 0.3).

Value

Invisibly returns `NULL`. Called for its side-effect of drawing arrows on the active plot device.

adjust.to.2pi *Wrap Angle to [0, 2pi)*

Description

Wrap Angle to [0, 2pi)

Usage

```
adjust.to.2pi(x)
```

Arguments

x	Numeric. Angle(s) in radians.

Value

Numeric. Angle(s) mapped to `[0, 2pi)`.

alpha.adjust *Alpha Adjustment for Multiple Testing*

Description

Alpha Adjustment for Multiple Testing

Usage

```
alpha.adjust(alpha, k, method = "bonferroni")
```

Arguments

alpha	Original alpha
k	Number of tests
method	"bonferroni" or "sidak"

Value

Adjusted alpha

amp.cut *Amplitude Cutoff Adjustment*

Description

Amplitude Cutoff Adjustment

Usage

```
amp.cut(amp, a.TOJR)
```

Arguments

amp	Vector indicating if amplitudes exceed cutoff
a.TOJR	Current TOJR category

Value

Adjusted TOJR category

bootstrapParams *Bootstrap parameter sets from pilot fit*

Description

Resamples G rows (genes) with replacement from param_df to generate nboot independent parameter draws.

Usage

```
bootstrapParams(param_df, nboot, seed = 42)
```

Arguments

param_df	Data frame from fitCosinorAll() (G rows)
nboot	Number of bootstrap draws
seed	Random seed

Value

List of length nboot; each element is a data frame like param_df

calculate_CI.M *Calculate CI for Mesor*

Description

Calculate CI for Mesor

Usage

```
calculate_CI.M(XX.inv, A.t, r.full = 6, n, alpha = 0.05, sigma.hat)
```

Arguments

XX.inv	Inverse of the design matrix cross-product (t(X) times X).
A.t	Contrast row vector selecting the mesor.
r.full	Residual degrees-of-freedom offset (default 6).
n	Number of observations.
alpha	Significance level (default 0.05).
sigma.hat	Estimated residual standard deviation.

Value

CI half-width (radius) for the mesor estimate.

calculate_CI_A.phase.Taylor

Calculate CI for Amplitude and Phase (Taylor method)

Description

Calculate CI for Amplitude and Phase (Taylor method)

Usage

```
calculate_CI_A.phase.Taylor(XX.inv, A.t, phase.hat, A.hat, sigma.hat)
```

Arguments

XX.inv	Inverse of the design matrix cross-product (t(X) times X).
A.t	Contrast row vector selecting the cosinor coefficients.
phase.hat	Estimated phase (radians).
A.hat	Estimated amplitude.
sigma.hat	Estimated residual standard deviation.

Value

List with `se.A.hat` and `se.phase.hat`, the delta-method standard errors for amplitude and phase.

choose.abs.min

Choose the minimum absolute value from each row

Description

Choose the minimum absolute value from each row

Usage

```
choose.abs.min(mat = matrix(stats::rnorm(21), ncol = 3))
```

Arguments

mat	Numeric matrix; one candidate value per column, one case per row.

Value

Numeric vector, length `nrow(mat)`: the smallest-magnitude entry from each row (sign preserved).

`CircadianAnalysisOptions` *Create Analysis and Reporting Options*

Description

Collects the settings that govern how each simulated dataset is analysed and how power is reported: the per-gene significance level, the multiple-testing correction, the FDR thresholds at which power curves are drawn, and the effect-size strata used to break power down by signal strength. The resulting `CircadianAnalysisOptions` object is passed to the `runSims*` and plotting functions.

Usage

```
CircadianAnalysisOptions(
  alpha = 0.05, p.adjust.method = "BH", parallel.ncores = 1,
  amp.cutoff = 0, target_effect = 0.1,
  fdr_thresholds = c(0.01, 0.05, 0.10, 0.20), reference_n = 60,
  r_strata = c(0, 0.25, 0.5, 0.75, 1, 1.25, 1.5, 1.75,
               2, 2.5, 3, 3.5, 4, 4.5, 5, Inf),
  strata_labels = NULL,
  phase_shifts = c(0, 0.5, 1, 2, 4, 6, 8, 10, 12),
  DCmethod = "DCP")
```

Arguments

`alpha`	Per-gene significance level for the cosinor rhythmicity test.
`p.adjust.method`	Multiple-testing correction applied across genes, passed to `stats::p.adjust` (default Benjamini-Hochberg).
`parallel.ncores`	Number of cores used to fit the per-gene tests.
`amp.cutoff`	Minimum amplitude a gene must exceed to enter the cosinor rhythmicity screen.
`target_effect`	Minimum effect size a gene must reach to count as a meaningful discovery.
`fdr_thresholds`	FDR thresholds at which power curves are reported.
`reference_n`	Reference sample size marked on the panel-B and phase-shift plots.
`r_strata`	Breakpoints, in $r = A/\sigma$ units, that define the effect-size strata used to report power by signal strength.
`strata_labels`	Labels for the strata; generated automatically from `r_strata` when `NULL`.
`phase_shifts`	Phase-shift magnitudes (hours) swept in the phase-sensitivity analysis.
`DCmethod`	Differential-rhythmicity detector to report against, either `"DCP"` (the cosinor F-test, default) or `"CircaCompare"`. `"DCP"` is a retained legacy alias for the cosinor detector.

Value

Object of class "CircadianAnalysisOptions"

Examples

```
# Reporting options: 5 percent alpha, power at two FDR thresholds.
aopt <- CircadianAnalysisOptions(alpha = 0.05,
                                 fdr_thresholds = c(0.05, 0.20))
aopt
```

`CircadianBioOptions` *Create Biology and Differential Options*

Description

Builds the `CircadianBioOptions` object that describes the simulated transcriptome: the number of genes, the proportion that are rhythmic, and the gene-level distributions of baseline expression, noise, amplitude, and phase. For two-group studies it also records the proportions of genes that differ in rhythmicity, phase, or mesor between groups. Most users obtain this object indirectly from `estCircadianParam` or `scp_load_pilot`; the constructor is exposed for building a configuration by hand.

Usage

```
CircadianBioOptions(
  ngenes = 5000,
  prop_rhythmic = NULL,
  period = 24,
  lBaselineExpr = NULL,
  lBaselineExpr2 = NULL,
  lOD = NULL,
  lOD2 = NULL,
  amplitude = NULL,
  amplitude2 = NULL,
  sigma_rhythmic = NULL,
  paired_sigma = FALSE,
  cts = NULL,
  cts2 = NULL,
  phase = "uniform",
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  mesor_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  alpha_pilot = NULL,
  sim.seed = 12345
)
```

Arguments

ngenes	Number of genes.
prop_rhythmic	Proportion of rhythmic genes. If NULL and a pilot dataset name is supplied to lBaselineExpr, the pilot's estimated proportion is used.
period	Circadian period in hours.
lBaselineExpr	Log baseline expression. Can be a numeric scalar (constant for all genes), a numeric vector (resampled to ngenes), a function f(n) returning n values, or a character string naming a built-in pilot dataset stored in data/. No default, must be supplied.
lBaselineExpr2	As lBaselineExpr but for group 2 (differential analyses only). NULL uses the same distribution as group 1.
lOD	Log residual noise: log(sigma), the per-gene noise around the cosinor fit (sigma = exp(lOD)). Same forms accepted as lBaselineExpr. No default, must be supplied.
lOD2	As lOD (the residual noise) but for group 2. NULL shares group 1 values.
amplitude	Amplitude distribution for rhythmic genes, given in any of the forms accepted for lBaselineExpr. This argument is required and has no default.
amplitude2	As amplitude but for group 2.
sigma_rhythmic	Optional numeric vector of per-gene noise values for rhythmic genes (same length as amplitude). When provided, amplitude and sigma are drawn jointly to preserve pilot A-sigma correlation.
paired_sigma	Logical. If TRUE and sigma_rhythmic is supplied, expand amplitude and sigma_rhythmic jointly by the same resampled gene index so their pilot-estimated pairwise correlation is preserved; if FALSE (default), amplitude and sigma are expanded independently.
cts	Numeric vector of sample collection times for passive design.
cts2	As cts for group 2.
phase	Phase distribution for rhythmic genes. "uniform" (default) or the same forms as lBaselineExpr.
prop_DR	Proportion with differential rhythmicity.
prop_DP	Proportion with differential phase.
prop_DM	Proportion with differential mesor (mean shift, both groups rhythmic).
phase_diff	Range of phase shift for DP genes c(min, max).
amp_diff	Range of amplitude ratio (unused; retained for interface compatibility).
mesor_diff	Range of mesor shift for DM genes c(min, max) (additive, log-scale units).
dp_shift_mode	"fixed" (use phase_diff[2]) or "uniform" (sample uniformly within phase_diff range).
dr_amp_scale	Scale factor for amplitude (A) to adjust DR strength.
dr_sigma_scale	Scale factor for sigma to adjust DR strength.
alpha_pilot	Rhythmicity pre-screen threshold used to define the pilot's rhythmic gene set (the value at which prop_rhythmic and the effect-size distribution were calibrated). Stored on the object for provenance and reporting; default NULL.
sim.seed	Random seed.

Value

Object of class "CircadianBioOptions".

Examples

```
# Build a synthetic biology configuration without pilot data.
bio <- CircadianBioOptions(ngenes = 300L, prop_rhythmic = 0.3,
                           lBaselineExpr = 5, lOD = -1, amplitude = 1)
bio
```

CircadianBootstrapOptions

Create Bootstrap Grid Options

Description

Collects the settings for a bootstrapped design-grid analysis, which quantifies how much the finite pilot contributes to uncertainty in power across combinations of time points (B) and replicates (m). It defines the grid of designs to evaluate and the number of bootstrap draws and inner simulations used to form the confidence bands. The resulting object is passed to runBootstrapDesignGrid.

Usage

```
CircadianBootstrapOptions(
  design_vector,
  B_values = c(4, 6, 8, 12, 24),
  m_values = NULL,
  N_values = NULL,
  nboot = 200,
  nsims_inner = 20,
  design = c("active", "passive"),
  seed = 42
)
```

Arguments

design_vector	Ordered vector of candidate time points to sample from; its length must be at least max(B_values).
B_values	Numbers of distinct time points to evaluate.
m_values	Replicates per time point. When NULL, derived as round(N / B).
N_values	Total per-group sample sizes to evaluate. When NULL, derived as B * m.
nboot	Number of bootstrap parameter draws in the outer uncertainty loop.
nsims_inner	Number of simulations run per bootstrap draw.
design	Sampling design, either "active" or "passive".
seed	Random seed.

Value

Object of class "CircadianBootstrapOptions"

Examples

```
# A small active-design B-vs-m grid over two sample sizes.
bopt <- CircadianBootstrapOptions(design_vector = seq(0, 22, by = 2),
                                  B_values = 4, N_values = c(24, 48),
                                  nboot = 3, nsims_inner = 4)
bopt
```

CircadianDesignOptions

Create Study Design Options

Description

Collects the sampling-design choices for a power analysis: the sample sizes to evaluate, the number of simulation replicates per size, and whether collection times are placed on an even grid (active design) or drawn from the pilot's time-of-day distribution (passive design). The resulting `CircadianDesignOptions` object is passed to the `runSims*` and `recommendDesign` functions.

Usage

```
CircadianDesignOptions(
  sample_sizes = c(10, 20, 40, 60, 80, 100),
  nsims = 100,
  design = c("active", "passive"),
  cts = NULL,
  B_values = NULL,
  test_types = c("DR", "DP", "DM")
)
```

Arguments

`sample_sizes`	Numeric vector of sample sizes to evaluate.
`nsims`	Number of simulation replicates run at each sample size.
`design`	Sampling design: `"active"` places collection times on an even grid, `"passive"` draws them from `cts`.
`cts`	Numeric sampling times, or a bundled pilot's `time_distribution` aggregate for a passive design.
`B_values`	Optional vector of time-point counts (B) to sweep, in addition to `sample_sizes`. `NULL` runs the single B implied by `cts`.
`test_types`	Character vector of differential tests to run, any of `"DR"`, `"DP"`, and `"DM"`.

Value

Object of class "CircadianDesignOptions"

Examples

```
# Active design over two small sample sizes with few simulations.
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
dopt
```

circaPowerApproxN80 *Analytical Sample Size for 80 percent Power*

Description

Returns a fast closed-form estimate of the sample size needed for the target power, evaluated at the pilot's median signal-to-noise ratio ($\tilde{r} = A/\sigma$). It uses the analytical cosinor F-test power formula rather than simulation, so it is a useful starting point before running the full simulation grid.

Usage

```
circaPowerApproxN80(
  bio.opts,
  alpha = 0.05,
  target_power = 0.8,
  n_search = seq(5, 500, by = 5)
)
```

Arguments

bio.opts	A CircadianBioOptions pilot; its amplitude and sigma_rhythmic give the signal-to-noise distribution.
alpha	Significance level for the per-gene test (default 0.05).
target_power	Target power to solve for (default 0.80).
n_search	Integer vector of candidate sample sizes to search over.

Value

A single integer: the smallest sample size in n_search that reaches target_power, or NA if none does.

Examples

```
# Analytical sample size for the pilot's median signal-to-noise ratio.
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
circaPowerApproxN80(bio)
```

circular_concentration *Circular concentration (1 - circular variance)*

Description

Circular concentration (1 - circular variance)

Usage

```
circular_concentration(angles, period = 24)
```

Arguments

angles	Angles (in hours)
period	Period (default 24)

Value

Concentration (0 = uniform, 1 = all same)

---

circular_difference *Signed circular difference between two phases*

Description

Signed circular difference between two phases

Usage

```
circular_difference(phi1, phi2, period = 24)
```

Arguments

phi1, phi2	Phases (same units as period, e.g. hours).
period	Period used to wrap the difference (default 24).

Value

phi1 - phi2 wrapped to [-period/2, period/2].

---

circular_mean *Circular mean calculation*

Description

Circular mean calculation

Usage

```
circular_mean(angles, period = 24)
```

Arguments

angles	Angles (in hours)
period	Period (default 24)

Value

Mean angle (in hours)

CP_OneGroup *Single Group Rhythmicity Analysis (Internal)*

Description

Single Group Rhythmicity Analysis (Internal)

Usage

```
CP_OneGroup(x1, period = 24, alpha = 0.05, CI = FALSE, p.adjust.method = "BH")
```

Arguments

x1	Group 1 data (list with data, time, gname components)
period	Period (default 24)
alpha	Significance threshold
CI	Calculate confidence intervals?
p.adjust.method	P-value adjustment method

Value

List with rhythmicity results

detect_cosinor *Unified cosinor rhythmicity detection (single- or multi-harmonic)*

Description

Canonical entry point for rhythmicity detection by K-harmonic cosinor regression. K = 1 is the single-harmonic cosinor F-test, identical to the single-harmonic case; K = 2 adds the 12-hour second harmonic and tests the joint harmonic block. By default the unshrunk exact nested F-test is used (matching the power derivations in the paper). Returns raw p-values; the caller applies FDR control.

Usage

```
detect_cosinor(expr, times, K = 1L, period = 24,
               ebayes = FALSE, mc.cores = 1L)
```

Arguments

expr	Gene x sample expression matrix
times	Numeric time vector (length = ncol(expr))
K	Harmonic order (default 1)
period	Period in hours (default 24)
ebayes	For K >= 2, moderate residual variance with empirical-Bayes shrinkage (default FALSE, the unshrunk exact F-test)
mc.cores	Cores for the K >= 2 fit (default 1)

Value

Numeric vector of raw p-values, length `nrow(expr)`

See Also

`estCircadianParam` for calibrating a pilot from the same cosinor fit.

Examples

```
# Rhythmicity p-values for a small simulated matrix (10 rhythmic genes).
set.seed(1)
times <- rep(seq(0, 22, by = 2), 2)
expr  <- matrix(rnorm(50 * length(times)), nrow = 50)
expr[1:10, ] <- expr[1:10, ] + 2 * cos(2 * pi * times / 24)
pvals <- detect_cosinor(expr, times, K = 1)
sum(p.adjust(pvals, "BH") < 0.05)
```

---

estCircadianParam *Estimate Circadian Pilot Parameters (Single-Cohort)*

Description

Fits a cosinor model per gene, ranks rhythmic genes by p-value, and returns empirical parameter distributions packaged as a `CircadianBioOptions` object suitable for downstream simulation and power analysis.

Usage

```
estCircadianParam(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

data	Numeric matrix of pilot expression (genes x samples). A data.frame is coerced to matrix.
times	Numeric vector of sample collection times in hours, length ncol(data). The function does NOT validate units; pass hours (0 to 24 scale).
period	Period in the same units as times (default 24 hours).
min_rhythm_pval	Uncorrected cosinor p-value threshold for the rhythmic-candidate set (default 0.01). Used to compute prop_rhythmic and to select the top-K (K = min(top_k, n_rhythmic)) genes whose amplitudes, phases, and noise drive simulation truth.
top_k	Number of highest-signal genes (ranked by cosinor p-value) that form the effect-size distribution (amplitude, phase, noise). Default 300. If top_k exceeds the number of rhythmic genes, it is capped to that number with a warning. Inf or "all" uses every rhythmic candidate.
prop_DR, prop_DP, prop_DM	Proportions of differentially-rhythmic, differentially-phased, and differentially-mesor genes (defaults 0.15, 0.10, 0.00). Capped automatically at prop_rhythmic.
mesor_diff, phase_diff, amp_diff	Numeric length-2 ranges for the differential effect-size sampling.
dp_shift_mode	One of "fixed" or "uniform" (default "fixed").
dr_amp_scale, dr_sigma_scale	Scaling factors for DR-class amplitude and noise distributions (default 1).
paired_sigma	Logical, default FALSE. If TRUE, joint (A, sigma) sampling preserves pilot correlation in downstream simulations.
sim.seed	Integer seed for downstream simulation (default 12345).
verbose	Logical, default TRUE. Print summary at end.

Value

A CircadianBioOptions S3 object usable directly with runSimsSingleCohort() or runDifferentialPower().

See Also

estCircadianParamTwoGroup for the two-group setup, prepCircadianData for preparing the input matrix.

Examples

```
set.seed(1)
n_genes <- 200; n_samples <- 30
times <- rep(seq(0, 22, by = 2), length.out = n_samples)
expr  <- matrix(rnorm(n_genes * n_samples), nrow = n_genes)
for (g in 1:20) expr[g, ] <- expr[g, ] + 2 * cos(2 * pi * times / 24)
bio <- estCircadianParam(expr, times, period = 24,
                         top_k = 20, verbose = FALSE)
```

`estCircadianParam2H` *Two-Harmonic Circadian Pilot Estimation*

Description

Two-harmonic analog of `estCircadianParam`. Fits the regression

$$y = \mu + a_1 \cos(\omega_0 t) + b_1 \sin(\omega_0 t) + a_2 \cos(2\omega_0 t) + b_2 \sin(2\omega_0 t) + \epsilon$$

to each gene by OLS, tests the joint rhythmicity null $H_0 : a_1 = b_1 = a_2 = b_2 = 0$ on 4 d.f., and returns a `CircadianBioOptions` object whose 5-way joint empirical distribution $F_{(\mu, A_1, \phi_1, A_2, \phi_2, \sigma)}$ is preserved by gene-paired index draws downstream.

The returned object carries the standard 1-harmonic fields (`amplitude`, `phase`, `sigma_rhythmic`) holding the 1st-harmonic $A_{g,1}$, $\phi_{g,1}$, and σ_g respectively, and three extra fields attached to the list after construction:

- `amplitude2`: 2nd-harmonic amplitudes $A_{g,2}$
- `phase2`: 2nd-harmonic acrophases $\phi_{g,2}$ in hours $[0, period/2)$
- `paired_2h`: `TRUE` flag (downstream simulators read this to switch on the 2-harmonic generative model).

These extras live alongside `$diagnostics`; the constructor is not modified.

Usage

```
estCircadianParam2H(
  data,
  times,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 500,
  prop_DR = 0.15,
  prop_DP = 0.1,
  prop_DM = 0,
  mesor_diff = c(0.5, 2),
  phase_diff = c(-6, 6),
  amp_diff = c(0.5, 2),
  dp_shift_mode = c("fixed", "uniform"),
  dr_amp_scale = 1,
  dr_sigma_scale = 1,
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

`data`	Numeric matrix of pilot expression (genes x samples).
`times`	Numeric vector of sample collection times in hours, length `ncol(data)`.
`period`	Period in the same units as `times` (default 24 hours).

`min_rhythm_pval`
: P-value threshold for the joint 4-d.f. F-test (default 0.01). Genes with p < this define the candidate set.

`top_k`
: Cap on number of genes contributing to the joint $(A_1, \phi_1, A_2, \phi_2, \sigma)$ distribution (default 500), ranked by F-test p-value.

`prop_DR`, `prop_DP`, `prop_DM`
: Differential proportions (defaults 0.15, 0.10, 0.00). Forwarded unchanged to `CircadianBioOptions`; note the K=2 extension itself is single-cohort only.

`mesor_diff`, `phase_diff`, `amp_diff`
: Differential effect-size ranges.

`dp_shift_mode`
: "fixed" or "uniform" (default "fixed").

`dr_amp_scale`, `dr_sigma_scale`
: DR scale factors (default 1).

`paired_sigma`
: Logical (default FALSE). Forwarded to `CircadianBioOptions`.

`sim.seed`
: Integer seed (default 12345).

`verbose`
: Logical (default TRUE). Print diagnostic summary.

Value

A `CircadianBioOptions` object with extra fields `$amplitude2`, `$phase2`, `$paired_2h`, and a `$diagnostics` list summarising the 2H fit (median A2/A1 ratio, fraction of genes with significant 2nd-harmonic block, etc.).

Methodological notes

1. Pilot summary is a 5-way joint empirical distribution. When the simulator draws a rhythmic gene, the same index j is used across $(A_1, \phi_1, A_2, \phi_2, \sigma)$, preserving the cross-parameter dependence observed in the pilot.
2. No 2nd-harmonic significance threshold is applied: ALL top-K genes passing the joint 4-d.f. F-test contribute, even those whose A_2 is essentially noise. The empirical F_{A_2} will have a noise spike near zero, which is fine.
3. The K=2 extension is single-cohort only; the differential simulator is not extended.

See Also

`estCircadianParam` for the 1-harmonic case, `simCircadianSingleCohort2H` for the matching simulator.

Examples

```
# Two-harmonic pilot fit on a bundled raw pilot (first 300 genes).
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                            package = "SCP"))
expr <- praw$adrenal$expr[1:300, ]
bio2h <- estCircadianParam2H(expr, praw$adrenal$times,
                             top_k = 100, verbose = FALSE)
bio2h$prop_rhythmic
```

`estCircadianParamTwoGroup` *Two-Group Circadian Pilot Estimation*

Description

Fits cosinor models to two groups of pilot data and assembles the two-group pilot summary used by the differential power simulator, encoding differential rhythmicity (DR), differential phase (DP), and differential MESOR (DM) structure.

Usage

```
estCircadianParamTwoGroup(
  data_1,
  data_2,
  times_1,
  times_2,
  period = 24,
  min_rhythm_pval = 0.01,
  top_k = 300L,
  phase_shift_threshold = 2,
  prop_DM = NULL,
  mesor_diff = NULL,
  dp_shift_mode = c("uniform", "fixed"),
  paired_sigma = FALSE,
  sim.seed = 12345,
  verbose = TRUE
)
```

Arguments

`data_1`, `data_2`	Gene-by-sample expression matrices for the two groups.
`times_1`, `times_2`	Numeric collection times for each group.
`period`	Period in hours (default 24).
`min_rhythm_pval`	Marginal p-value screen defining the rhythmic pilot set.
`top_k`	Number of highest-signal genes (per group, ranked by cosinor p-value) forming each group's effect-size distribution (default 300). Capped (with a warning) to a group's rhythmic-gene count if larger; `Inf` or `"all"` uses every rhythmic candidate.
`phase_shift_threshold`	Minimum phase difference (hours) for the DP class.
`prop_DM`	Optional target proportion of differential-MESOR genes.
`mesor_diff`	Optional MESOR difference for the DM component.
`dp_shift_mode`	Differential-phase shift model, "uniform" or "fixed".
`paired_sigma`	Preserve the paired (amplitude, noise) relationship across groups.
`sim.seed`	Random seed.
`verbose`	Print progress.

Value

A two-group `CircadianBioOptions` pilot summary.

Examples

```
# Two-group pilot (adrenal vs liver arms of the bundled demo pilot).
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                            package = "SCP"))
bio <- estCircadianParamTwoGroup(praw$adrenal$expr[1:300, ],
                                 praw$liver$expr[1:300, ],
                                 praw$adrenal$times, praw$liver$times,
                                 top_k = 100, verbose = FALSE)
bio$prop_DR
```

`fishers.p` *Fisher's Combined P-value*

Description

Fisher's Combined P-value

Usage

```
fishers.p(ps)
```

Arguments

`ps`	Vector of p-values

Value

Combined p-value

`forwardStop` *Forward Stopping Rule*

Description

Forward Stopping Rule

Usage

```
forwardStop(pv, alpha = 0.1)
```

Arguments

`pv`	P-values
`alpha`	Threshold

Value

List with stop and p

get_phase *Get Phase from Beta Coefficients*

Description

Get Phase from Beta Coefficients

Usage

```
get_phase(b1.x, b2.x)
```

Arguments

b1.x	Cosine-term coefficient.
b2.x	Sine-term coefficient.

Value

List with phase (radians) and tan (the ratio used).

launchShiny *Launch the SCP Shiny app*

Description

Opens the bundled simulation-based power-analysis app in the user's default browser. The app lets the user pick a bundled pilot (or upload their own), set target power and FDR, read off the recommended sample size, explore per-gene rhythmicity, and run pathway enrichment. Bundled GTEx datasets use v10 expression with season-adjusted time of death (TOD).

Usage

```
launchShiny(..., install_deps = TRUE, species_all = FALSE)
```

Arguments

...	Additional arguments passed to shiny::runApp().
install_deps	Logical; if TRUE (default) install any missing optional app packages before launching. Set FALSE to skip the check (missing features then degrade gracefully inside the app).
species_all	Logical; if TRUE also install the annotation databases for rat, zebrafish, Drosophila, and Arabidopsis (in addition to human + mouse) so upload gene-symbol mapping works for every supported species. Default FALSE installs only human + mouse (the common case); the others install on demand or you can pass TRUE once.

Details

On first launch the function checks for the app's optional helper packages and offers to install any that are missing (so the gene-symbol mapping, XLSX export, full-app capture, and pathway-enrichment features work out of the box).

Value

Invisibly returns `NULL`. Called for its side effect of launching the Shiny app.

Examples

```
## Not run:
  launchShiny()
  launchShiny(species_all = TRUE)    # add rat/fish/fly/plant DBs
  launchShiny(install_deps = FALSE)  # skip the dependency check

## End(Not run)
```

LR_diff *Likelihood Ratio Test for Differential Parameters*

Description

Likelihood Ratio Test for Differential Parameters

Usage

```
LR_diff(time1, y1, time2, y2, period, FN = TRUE, type = "amplitude")
```

Arguments

`time1`	Time points for condition 1
`y1`	Expression values for condition 1
`time2`	Time points for condition 2
`y2`	Expression values for condition 2
`period`	Period
`FN`	Finite sample correction
`type`	Parameter type: "amplitude", "phase", "basal"

Value

List with p-value and parameter estimates

makeAdaptiveRStrata *Adaptive r-Strata Breakpoints*

Description

Generates breakpoints from 0 to ceiling(r_max / bin_width) * bin_width, stepping by bin_width, then appends Inf. Every bin width is identical, but the number of bins adapts to the empirical r range of the pilot data.

Usage

```
makeAdaptiveRStrata(
  bio.opts,
  bin_width = 0.25,
  r_min_pct = 0,
  r_pctile_cap = 0.99
)
```

Arguments

bio.opts	CircadianBioOptions with amplitude and sigma_rhythmic.
bin_width	Width of each r bin (default 0.25).
r_min_pct	Lower percentile of pilot r used as range floor (default 0 = always start from 0).
r_pctile_cap	Upper percentile of pilot r used as the range ceiling before rounding up to the nearest bin_width (default 0.99).

Value

Numeric vector of breakpoints starting at 0 and ending at Inf.

Examples

```
# Adaptive r = A/sigma strata from a bundled pilot.
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
makeAdaptiveRStrata(bio, bin_width = 0.5)
```

npower *Sample Size for a Target Genome-Wide Power*

Description

Works on output from either runSimsSingleCohort() or runSimsDiff(). Recomputes marginal power at the requested FDR from raw simulation output, then returns the smallest grid N that meets the target (n_grid) and, when interpolate = TRUE (the default), a linearly interpolated estimate between the two bracketing grid points (n).

Usage

```
npower(
  res,
  target_power = 0.8,
  fdr = 0.05,
  endpoint = NULL,
  p.adjust.method = "BH",
  interpolate = TRUE
)
```

Arguments

res	Output from runSimsSingleCohort() or runSimsDiff().
target_power	Numeric in (0, 1). Target marginal power (default 0.80).
fdr	FDR threshold (default 0.05).
endpoint	For runSimsDiff() output: one of "DR", "DP", "DM". Ignored for single-cohort output.
p.adjust.method	Multiple-testing correction (default "BH"). Only applied for single-cohort output (differential uses pre-computed FDR arrays).
interpolate	Logical. If TRUE (default), linearly interpolates between the two adjacent grid points to return a more precise n estimate. Set to FALSE to revert to the grid-snapped n_grid value (backward-compatible).

Value

An object of class "npower" (a list) with:

n	Recommended sample size: n_interp when interpolate = TRUE, otherwise n_grid. NA if target power is never reached.
n_grid	Smallest sample size in the simulation grid achieving target power, or NA.
n_interp	Linearly interpolated sample size between the two bracketing grid points (a fractional N rounded up). NA when interpolate = FALSE or when target power is never reached.
power	Named numeric vector: mean power at each grid sample size.
target_power, fdr, endpoint	Echo of inputs.

Examples

```
bio  <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res  <- runSimsSingleCohort(bio, dopt, aopt, verbose = FALSE)
np <- npower(res, target_power = 0.8, fdr = 0.05)
np
```

one_cosinor_OLS *Fit a Cosinor Model to a Single Gene by OLS*

Description

Fits the cosinor model $y = M + A\cos(\omega t - \phi) + \epsilon$ via ordinary least squares by reparameterising as $y = M + \beta_1\cos(\omega t) + \beta_2\sin(\omega t) + \epsilon$. Returns mesor, amplitude, acrophase, p-value, and R^2.

Usage

```
one_cosinor_OLS(tod, y, period = 24, compute.phase.CI = FALSE, CI.level = 0.95)
```

Arguments

tod	Numeric vector. Sample time points (hours).
y	Numeric vector. Gene expression values (same length as tod).
period	Numeric. Rhythm period in hours (default 24).
compute.phase.CI	Logical. Reserved; not yet implemented (default FALSE).
CI.level	Numeric. Confidence level for phase CI (default 0.95).

Value

Named list with fields: M (mesor), A (amplitude), phi (acrophase in hours), pvalue (F-test p-value), R2 (coefficient of determination). Returns NA for all fields if the design matrix is singular or $n \leq 3$.

plot.SCPDiffResult *Plot an SCPDiffResult object*

Description

Plot an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPDiffResult from runDifferentialPower().
output_file	Optional PDF path; NULL draws to the active device.
...	Ignored.

Value

x, invisibly.

plot.SCPRecommendResult *Plot an SCPRecommendResult object*

Description

Plot an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPRecommendResult from recommendDesign().
output_file	Optional PDF path; NULL draws to the active device.
...	Passed to the underlying plot method.

Value

x, invisibly.

plot.SCPSingleResult *Plot an SCPSingleResult object*

Description

Plot an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
plot(x, output_file = NULL, ...)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid().
output_file	Optional PDF path; NULL draws to the active device.
...	Ignored.

Value

x, invisibly.

plotBootstrapDesignGrid *Plot bootstrap design grid results*

Description

Panel A: Power vs N, one line per B, with bootstrap CI ribbon. Panel B: Heatmap of mean power over (N, B) grid.

Usage

```
plotBootstrapDesignGrid(
  result,
  test_type = "DR",
  fdr_threshold = NULL,
  panels = "A",
  output_file = NULL
)
```

Arguments

result	Output from runBootstrapDesignGrid()
test_type	Test type to plot
fdr_threshold	FDR threshold (label only)
panels	Which panel(s) to draw: "A" (power vs N) and/or "B" (heatmap) (default "A").
output_file	Path for PDF output (NULL = screen)

Examples

```
praw  <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                             package = "SCP"))
expr  <- praw$adrenal$expr[1:300, ]
times <- praw$adrenal$times
bio   <- estCircadianParam(expr, times, verbose = FALSE)
bopt  <- CircadianBootstrapOptions(
           design_vector = sort(unique(round(times))),
           B_values = 4, N_values = c(24, 48),
           nboot = 3, nsims_inner = 4, design = "passive")
aopt  <- CircadianAnalysisOptions(alpha = 0.05)
grid  <- runBootstrapDesignGrid(expr, times, bopt, aopt,
                                bio_diff.opts = bio, mode = "single",
                                verbose = FALSE)
plotBootstrapDesignGrid(grid, test_type = "rhythmic",
                        output_file = tempfile(fileext = ".pdf"))
```

`plotDiffPower` *Plot differential circadian power*

Description

Plots two-group differential power for one or more comparisons from the objects returned by `runDifferentialPower`. By default it draws marginal power against sample size, with one column per endpoint and one row per comparison. Setting `stratified = TRUE` adds the effect-size-stratified panels (power and true discoveries by effect size) in a three-column layout.

Usage

```
plotDiffPower(
  res_list,
  comp_labels = names(res_list),
  endpoints = c("DR", "DP", "DM"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  display_sizes = NULL,
  stratified = FALSE,
  r_break_width = 0.25,
  r_max = 5,
  r_display_max = NULL,
  main_title = "Differential Circadian Power Analysis",
  out_pdf = NULL,
  width = NULL,
  height = NULL
)
```

Arguments

`res_list`	Named list of length 2, each element output of runSimsDiff().
`comp_labels`	Character(2), display labels for the two comparisons.
`endpoints`	Which endpoints to include (default c("DR","DP","DM")).
`fdr_thresholds`	FDR levels for panel A (default c(0.01,0.05,0.10,0.20)).
`panel_fdr`	FDR threshold used to draw the marginal power curve in the non-stratified layout (default 0.05).
`vline_power`	Horizontal reference line for target power, drawn at this value (default 0.80).
`vline_fdr`	FDR threshold marked with a reference guide on the power panels (default 0.20).
`display_sizes`	Sample sizes to show in all panels (NULL = all).
`stratified`	Logical (default FALSE). If FALSE, draws only the marginal power-vs-sample-size panel per endpoint (1 x n_endpoints layout per comparison). If TRUE, also draws power-by-r and true-discoveries-by-r strata panels in the original 3-column layout.
`r_break_width`	Width of r-strata bins (default 0.25). Ignored if stratified = FALSE.

r_max	Upper bound (in r = A/sigma units) for the r-strata breakpoints when stratified = TRUE (default 5).
r_display_max	Optional upper x-axis limit (r units) for the stratified panels; NULL (default) uses the full stratified range.
main_title	Outer plot title (default "Differential Circadian Power Analysis").
out_pdf	Path for PDF. If NULL, plots to current device.
width	PDF width in inches. Defaults adapt to stratified.
height	PDF height in inches. Defaults adapt to stratified.

Value

Invisibly, list of panel data per comparison/endpoint.

Examples

```
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                            package = "SCP"))
e1 <- praw$adrenal$expr[1:300, ]; t1 <- praw$adrenal$times
e2 <- praw$liver$expr[1:300, ];   t2 <- praw$liver$times
bio  <- estCircadianParamTwoGroup(e1, e2, t1, t2, top_k = 100,
                                  verbose = FALSE)
bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res  <- runDifferentialPower(bio, dopt, aopt, test_types = "DR",
                             plot = FALSE, verbose = FALSE)
plotDiffPower(list(res), endpoints = "DR",
              out_pdf = tempfile(fileext = ".pdf"))
```

---

plotSingleCohortPower *Plot single-cohort circadian power (3 panels)*

Description

Summarises a single-cohort power simulation in three panels: marginal power against sample size at several FDR thresholds (Panel A), power resolved by effect-size stratum (Panel B), and the number of true discoveries by effect-size stratum (Panel C). It either draws to the current device or writes a PDF.

The panels can be produced two ways. Supply a precomputed runSimsSingleCohort object as res (optionally with a second object as panel_a_res that drives only the marginal panel). Alternatively, supply the pilot and design directly through bio.opts and design.opts: the marginal panel (A) is then computed from a run over the pilot's own effect sizes, while the stratified panels (B and C) are computed from a companion run that spreads gene effect sizes across the full $r = A/\sigma$ range so every stratum is populated. The caller never specifies how effect sizes are drawn.

Usage

```
plotSingleCohortPower(
  res = NULL,
  out_pdf = NULL,
  title = "",
  panel_a_res = NULL,
  panels = c("A", "B", "C"),
  panel_labels = c("A", "B", "C"),
  fdr_thresholds = c(0.01, 0.05, 0.1, 0.2),
  panel_fdr = 0.05,
  vline_power = 0.8,
  vline_fdr = 0.2,
  fdr = NULL,
  p.adjust.method = "BH",
  reference_n = NULL,
  display_sizes = NULL,
  r_max = 5,
  cex_main = 1.5,
  cex_lab = 1.6,
  cex_axis = 1.45,
  line_lwd = 2,
  pt_cex = 0.95,
  legend_cex = NULL,
  title_cex = 1.65,
  oma_top = 2.4,
  legend_pos = "bottomright",
  se_cap = 0.3,
  vertical = FALSE,
  panel_c_legend = FALSE,
  width = 15,
  height = 5.5,
  bio.opts = NULL,
  design.opts = NULL,
  analysis.opts = NULL,
  K = 1L,
  mc.cores = 1L
)
```

Arguments

res	Output from runSimsSingleCohort. May be NULL when bio.opts and design.opts are supplied, in which case the required simulations are run internally.
out_pdf	Path for PDF output. If NULL, plots to current device.
title	Overall figure title (default empty).
panel_a_res	Optional alternate result object used for the Panel A marginal-power computation only. If NULL (default), Panel A is computed from res. Useful when the marginal summary should reflect a different empirical assumption than the effect-size-stratified panels.
panels	Character vector, which of panels "A" (marginal power vs n), "B" (power by effect-size stratum), "C" (true discoveries by effect-size stratum) to draw (default all three).

panel_labels	Character vector of length 3 giving the letters printed on panels A, B, C respectively (default c("A","B","C")). Use e.g. c("D","E","F") for the lower figure in a vertically stacked pair.
fdr_thresholds	FDR levels for panel A (default c(0.01,0.05,0.10,0.20)).
panel_fdr	FDR threshold highlighted in panels B/C (default 0.05).
vline_power	Horizontal reference line for target power (default 0.80).
vline_fdr	FDR threshold marked with a reference guide (default 0.20).
fdr	Single-value alias that sets both panel_fdr and vline_fdr at once (back-compat convenience; default NULL).
p.adjust.method	Correction method (default "BH").
reference_n	Reference sample size for annotations (default NULL = none).
display_sizes	Sample sizes to show in all panels (NULL = all).
r_max	Upper bound (r = A/sigma units) for the r-strata breakpoints; NULL adapts to the 95th percentile of observed r (default 5).
cex_main	Title character expansion (default 1.50).
cex_lab	Axis-label character expansion (default 1.60).
cex_axis	Axis-tick character expansion (default 1.45).
line_lwd	Line width for power curves (default 2).
pt_cex	Point character expansion for plotted markers (default 0.95).
legend_cex	Legend text character expansion; NULL (default) picks a size automatically.
title_cex	Outer-title character expansion (default 1.65).
oma_top	Outer margin (lines) reserved above the panels for the title (default 2.4).
legend_pos	Legend position keyword passed to legend() (default "bottomright").
se_cap	Maximum half-width (power units) drawn for standard-error bars, to keep long bars from overplotting neighboring panels (default 0.3).
vertical	Logical; if TRUE, stack panels vertically instead of the default horizontal 1 x length(panels) layout (default FALSE).
panel_c_legend	Logical; if TRUE, also draw the legend on panel C (default FALSE; panel A's legend is normally sufficient).
width	PDF width in inches (default 15).
height	PDF height in inches (default 5.5).
bio.opts	Optional CircadianBioOptions pilot (e.g. from estCircadianParam). When supplied together with design.opts, and panel_a_res is NULL, the simulations backing the requested panels are run internally so the caller can go straight from a pilot to the figure. Ignored when panel_a_res is supplied (default NULL).
design.opts	Optional CircadianDesignOptions used with bio.opts for the internal runs (default NULL).
analysis.opts	Optional CircadianAnalysisOptions for the internal runs; defaults to CircadianAnalysisOptions when NULL.
K	Cosinor harmonic order for the internal runs (1 = single-harmonic, 2 = two-harmonic; default 1).
mc.cores	Cores used for the internal simulation runs (default 1).

Value

Invisibly returns list of data used per panel.

Examples

```
bio  <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res  <- runSimsSingleCohort(bio, dopt, aopt, verbose = FALSE)
plotSingleCohortPower(res, out_pdf = tempfile(fileext = ".pdf"))
```

---

prepCircadianData *Prepare expression data for circadian power analysis*

Description

Converts raw expression data into the standard (matrix, times) format used by the pilot-calibration and power functions (for example `estCircadianParam` and `runBootstrapDesignGrid`).

The pipeline always expects:

- `data`, numeric matrix, **genes x samples**, log2-scale
- `times`, numeric vector, hours in [0, 24), one per column of `data`

This function handles the three most common raw formats:

"counts" Raw integer counts (e.g. featureCounts, STAR); converted to `log2(CPM + 1)`.

"cpm" Raw CPM values (e.g. edgeR, baboon data); converted to `log2(CPM + 1)`.

"log2" Already log2-scale (e.g. GSE54651, Seney ACC); wrapped with `data.matrix()`, no numeric transformation.

Usage

```
prepCircadianData(
  expr,
  times,
  input_type = c("counts", "cpm", "log2"),
  pheno = NULL,
  sample_col = NULL,
  time_col = NULL,
  period = 24,
  verbose = TRUE
)
```

Arguments

`expr`	A matrix, data.frame, or file path (CSV/TSV) of expression values. Rows = genes, columns = samples. Row names used as gene IDs.
`times`	Numeric vector of time-of-day values (hours, 0-24) for each sample, OR a character column name in `pheno` to extract times from.

input_type	One of "counts", "cpm", or "log2". Default "counts".
pheno	Optional data.frame of sample metadata. Must have the same column order as expr if times is a column name string.
sample_col	Optional: name of the column in pheno whose values match colnames(expr) for alignment. If NULL, assumes columns are already in the same order as rows of pheno.
time_col	Name of the TOD column in pheno (used when times is a column name string).
period	Circadian period in hours. Default 24. Used only for validation.
verbose	Print a short summary of what was done. Default TRUE.

Value

A list with:

data	Numeric matrix (genes x samples), log2-scale
times	Numeric vector of hours (0-24), length == ncol(data)
n_genes	Number of genes
n_samples	Number of samples
input_type	The input_type used

Examples

```
# Wrap a bundled log2 pilot matrix into the (data, times) format.
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                            package = "SCP"))
expr <- praw$adrenal$expr[1:200, ]
tod  <- praw$adrenal$times
prep <- prepCircadianData(expr, tod, input_type = "log2")
str(prep[c("n_genes", "n_samples", "input_type")])
```

---

print.CircadianAnalysisOptions

Print a CircadianAnalysisOptions object

Description

Print a CircadianAnalysisOptions object

Usage

```
## S3 method for class 'CircadianAnalysisOptions'
print(x, ...)
```

Arguments

x	A CircadianAnalysisOptions object.
...	Ignored.

Value

x, invisibly.

print.CircadianBioOptions *Print a CircadianBioOptions object*

Description

Print a CircadianBioOptions object

Usage

```
## S3 method for class 'CircadianBioOptions'
print(x, ...)
```

Arguments

x	A CircadianBioOptions object.
...	Ignored.

Value

x, invisibly.

print.CircadianBootstrapOptions *Print a CircadianBootstrapOptions object*

Description

Print a CircadianBootstrapOptions object

Usage

```
## S3 method for class 'CircadianBootstrapOptions'
print(x, ...)
```

Arguments

x	A CircadianBootstrapOptions object.
...	Ignored.

Value

x, invisibly.

print.CircadianDesignOptions *Print a CircadianDesignOptions object*

Description

Print a CircadianDesignOptions object

Usage

```
## S3 method for class 'CircadianDesignOptions'
print(x, ...)
```

Arguments

x	A CircadianDesignOptions object.
...	Ignored.

Value

x, invisibly.

print.npower *Print an npower result*

Description

Print an npower result

Usage

```
## S3 method for class 'npower'
print(x, ...)
```

Arguments

x	An npower object (recommended N and its power curve).
...	Ignored.

Value

x, invisibly.

print.SCPDiffResult *Print an SCPDiffResult object*

Description

Print an SCPDiffResult object

Usage

```
## S3 method for class 'SCPDiffResult'
print(x, ...)
```

Arguments

x	An SCPDiffResult from runDifferentialPower().
...	Ignored.

Value

x, invisibly.

print.SCPRecommendResult *Print an SCPRecommendResult object*

Description

Print an SCPRecommendResult object

Usage

```
## S3 method for class 'SCPRecommendResult'
print(x, ...)
```

Arguments

x	An SCPRecommendResult from recommendDesign().
...	Ignored.

Value

x, invisibly.

print.SCPSingleResult *Print an SCPSingleResult object*

Description

Print an SCPSingleResult object

Usage

```
## S3 method for class 'SCPSingleResult'
print(x, ...)
```

Arguments

x	An SCPSingleResult from runSingleCohortGrid().
...	Ignored.

Value

x, invisibly.

rayleigh_test_circular *Rayleigh test for uniform circular distribution*

Description

Rayleigh test for uniform circular distribution

Usage

```
rayleigh_test_circular(angles, period = 24)
```

Arguments

angles	Angles (in hours)
period	Period (default 24)

Value

List with statistic and p-value

recommendDesign *B vs m design study (analytical and simulation)*

Description

Compares study designs that trade the number of time points (B) against the number of replicates per time point (m), and recommends a sample size for a target power. It first reports how each detector responds to B versus m, then computes analytical power at each sample size for the cosinor detector (which is invariant to B), and finally runs the full simulation for detectors that have no closed form. A previous result can be supplied to reuse its simulation instead of running it again.

Usage

```
recommendDesign(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = c("DCP", "JTK", "RAIN", "MH"),
  target_power = 0.8,
  mode = c("single", "differential"),
  run_simulation = TRUE,
  prior_result = NULL,
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

bio.opts	CircadianBioOptions from estCircadianParam() or estCircadianParamTwoGroup()
design.opts	CircadianDesignOptions with sample_sizes and B_values
analysis.opts	CircadianAnalysisOptions
methods	Detection methods to compare. Single-cohort: c("DCP","JTK","RAIN","MH"). Differential: c("DCP","CircaCompare","LimoRhyde","DODR").
target_power	Target power for recommendation (default 0.80)
mode	"single" or "differential" (default "single")
run_simulation	TRUE = run full simulation after analytical step. FALSE = analytical (CircaPower) only.
prior_result	A previous runSingleCohortGrid()/runDifferentialPower() result to reuse instead of re-running simulation.
alpha2	2nd-harmonic deviation (scalar or vector)
alpha3	3rd-harmonic deviation (scalar or vector)
mc.cores	Parallel cores for simulation step
plot	Auto-plot final recommendation
output_file	PDF path (NULL = screen)
verbose	Print progress

Value

An SCPRecommendResult object (a list) with:

- guidance: the method-guidance table.
- analytical_df: analytical power at each sample size (cosinor detector only).
- simulation: the simulation result, or NULL when no simulation was run.
- recommendation: the recommended design and sample size per method.

Examples

```
# Analytical (CircaPower) design guidance, no simulation.
bio  <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48, 96), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
rec  <- recommendDesign(bio, dopt, aopt, methods = "DCP",
                        run_simulation = FALSE, plot = FALSE,
                        verbose = FALSE)
rec$analytical_df
```

---

runBootstrapDesignGrid

Run bootstrap design grid simulation

Description

Estimates uncertainty in power across (N, B) design configurations by bootstrapping parameter sets from pilot data.

Usage

```
runBootstrapDesignGrid(
  pilot_data,
  pilot_times,
  boot.opts,
  analysis.opts,
  bio_diff.opts,
  pilot_data_2 = NULL,
  pilot_times_2 = NULL,
  mode = NULL,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  min_rhythm_pval = 0.01,
  resample = c("subject", "gene"),
  verbose = TRUE,
  mc.cores = 1L
)
```

Arguments

pilot_data	Genes x samples matrix (pilot experiment).
pilot_times	Numeric vector of sample time points, length = ncol(pilot_data).
boot.opts	The bootstrap-grid options built with CircadianBootstrapOptions(). Must set B_values to a single value for passive designs.
analysis.opts	CircadianAnalysisOptions.
bio_diff.opts	CircadianBioOptions from estCircadianParam(). Required even in mode = "single", differential proportion fields are read from here.
pilot_data_2	Optional group-2 pilot matrix for mode = "differential".
pilot_times_2	Optional group-2 pilot times.
mode	"single" or "differential".
methods	Detection method(s): "DCP" (default), "JTK", "RAIN".
test_types	Endpoints for differential mode: any of "DR", "DP", "DM".
alpha2	Second-harmonic coefficient (default 0).
alpha3	Third-harmonic coefficient (default 0).
min_rhythm_pval	P-value threshold for pilot rhythmicity (default 0.01).
resample	Resampling scheme for the outer bootstrap. "subject" (default) is the Efron nonparametric bootstrap over pilot subjects: each draw resamples subject columns with replacement (carrying their collection times) and refits the pilot summary, matching the Section 2.3 description. "gene" is the legacy scheme that fits once and resamples gene rows of the fit; retained for backward compatibility and diagnostics only.
verbose	Print progress (default TRUE).
mc.cores	Parallel cores for inner simulation loop (default 1).

Value

Named list with 15 elements:

N_values Candidate sample sizes tested.

B_values Candidate time-point counts tested.

m_matrix Matrix of total samples (N x B).

nboot Number of bootstrap replicates.

design Design type ("active" or "passive").

test_types Endpoints evaluated.

fdr_threshold FDR threshold used.

boot_power 4-D array [nboot x n_N x n_B x n_tests].

power_mean Mean power array [n_N x n_B x n_tests].

power_se Bootstrap SE array, same shape as power_mean.

power_ci_lo 2.5th percentile CI array.

power_ci_hi 97.5th percentile CI array.

power_plugin Plug-in (point-estimate) power array [n_N x n_B x n_tests] from the full pilot fit without resampling; overlaid as the plug-in line by plotBootstrapDesignGrid().

optimal_B Integer vector (length n_N): B with highest mean power at each N.

optimal_B_ci_lo Bootstrap 2.5th percentile of optimal B.

optimal_B_ci_hi Bootstrap 97.5th percentile of optimal B.

Examples

```
praw  <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                             package = "SCP"))
expr  <- praw$adrenal$expr[1:300, ]
times <- praw$adrenal$times
bio   <- estCircadianParam(expr, times, verbose = FALSE)
bopt  <- CircadianBootstrapOptions(
           design_vector = sort(unique(round(times))),
           B_values = 4, N_values = c(24, 48),
           nboot = 3, nsims_inner = 4, design = "passive")
aopt  <- CircadianAnalysisOptions(alpha = 0.05)
grid  <- runBootstrapDesignGrid(expr, times, bopt, aopt,
                                bio_diff.opts = bio, mode = "single",
                                verbose = FALSE)
grid$power_mean
```

---

runDifferentialPower *Run two-group differential circadian power analysis*

Description

Estimates FDR-controlled power to detect differential rhythmicity between two groups. For each sample size and simulation replicate it generates paired two-group data from the pilot, applies the chosen differential detector, and aggregates power for the requested endpoints (differential rhythmicity, phase, or mesor). Unless `plot = FALSE`, it also draws the power curves and prints the recommended sample size on completion.

Usage

```
runDifferentialPower(
  bio.opts,
  design.opts,
  analysis.opts,
  methods = "DCP",
  test_types = c("DR", "DP", "DM"),
  alpha2 = 0,
  alpha3 = 0,
  mc.cores = 1L,
  plot = TRUE,
  output_file = NULL,
  verbose = TRUE
)
```

Arguments

`bio.opts`	Two-group pilot from `estCircadianParamTwoGroup()`.
`design.opts`	The `CircadianDesignOptions` object (sample sizes, design, collection times).
`analysis.opts`	The `CircadianAnalysisOptions` object.
`methods`	Detectors to run: `"DCP"` (the cosinor F-test) or `"CircaCompare"`.

test_types	Endpoints to test, any of "DR" (rhythmicity), "DP" (phase), and "DM" (mesor). An endpoint a detector does not support is returned as NA.
alpha2	Scalar or vector swept for both groups
alpha3	Scalar or vector swept for both groups
mc.cores	Number of cores used for the per-gene fits.
plot	Draw the power curves when the run finishes.
output_file	Path for a PDF of the plot, or NULL to draw to the current device.
verbose	Print progress messages while running.

Value

A list of class SCPDiffResult. It holds the p-value and FDR arrays pval_DR, fdr_DR, pval_DP, fdr_DP, pval_DM, and fdr_DM, each indexed by gene, sample size, and simulation, together with diff_type, effectsize, sample_sizes, and nsims. Pass it to plotDiffPower() or to npower(..., endpoint = "DR").

Examples

```
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                            package = "SCP"))
e1 <- praw$adrenal$expr[1:300, ]; t1 <- praw$adrenal$times
e2 <- praw$liver$expr[1:300, ];   t2 <- praw$liver$times
bio  <- estCircadianParamTwoGroup(e1, e2, t1, t2, top_k = 100,
                                  verbose = FALSE)
bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res  <- runDifferentialPower(bio, dopt, aopt, test_types = "DR",
                             plot = FALSE, verbose = FALSE)
npower(res, endpoint = "DR", fdr = 0.05)$n
```

---

runSimsSingleCohort *Single-Cohort Rhythmicity Power Simulation*

Description

Extends the closed-form CircaPower approach by running full simulations: for each sample size and simulation replicate, generates data from the empirical pilot parameter distributions, applies the cosinor F-test per gene, applies BH correction, and aggregates empirical power and FDR.

Usage

```
runSimsSingleCohort(
  bio.opts,
  design.opts,
  analysis.opts,
  method = "cosinor",
  K = 1L,
  verbose = TRUE,
  mc.cores = 1L,
  progress = NULL
)
```

Arguments

bio.opts	The CircadianBioOptions pilot object, from estCircadianParam().
design.opts	The CircadianDesignOptions object (sample sizes, nsims, design).
analysis.opts	The CircadianAnalysisOptions object (alpha, correction, FDR thresholds).
method	Detection method: "cosinor" (default, the K-harmonic cosinor F-test, with K set by the K argument), or the alternative detectors "JTK", "RAIN", and "MH" (multi-harmonic). "DCP" and "Kharmonic" are retained aliases of the cosinor detector.
K	Integer. Number of harmonics for the cosinor detector (default 1). K=1 is the single-harmonic cosinor F-test; K=2 captures the 1st and 2nd harmonics; K=3+ for very sharp peaks.
verbose	Print progress (default TRUE).
mc.cores	Parallel cores (default 1).
progress	Optional callback function(sample_size_index, n_sample_sizes) invoked after each sample size completes (used by the Shiny app to advance its progress bar). NULL (default) disables the callback.

Details

Simulation-based alternative to the closed-form CircaPower formula: works for any design (active or passive) and any pilot parameter distribution.

Value

List with:

marginal_power	Matrix [sample_sizes x nsims]
marginal_FDR	Matrix [sample_sizes x nsims]
marginal_TD	Matrix [sample_sizes x nsims]
marginal_FD	Matrix [sample_sizes x nsims]
marginal_alpha	Matrix [sample_sizes x nsims]: empirical type-I error
pvalues	Array [sample_sizes x ngenes x nsims]: raw p-values
r_values_list	List[[sample_size]][[sim]]: per-gene A/sigma
strat_power	Array [sample_sizes x n_strata x nsims]
strat_TD	Array [sample_sizes x n_strata x nsims]
strat_FD	Array [sample_sizes x n_strata x nsims]
strat_n_targets	Array [sample_sizes x n_strata x nsims]
strat_n_nulls	Array [sample_sizes x n_strata x nsims]
strat_FDR	Array [sample_sizes x n_strata x nsims]
strat_alpha	Array [sample_sizes x n_strata x nsims]
r_strata	Stratum breakpoints (from analysis.opts)
strata_labels	Stratum labels
n0_circapower	CircaPower n80 estimate from pilot median r
sample_sizes	Vector of sample sizes
nsims	Number of simulation replicates

Examples

```
bio  <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio$ngenes <- 200L
dopt <- CircadianDesignOptions(sample_sizes = c(24, 48), nsims = 5)
aopt <- CircadianAnalysisOptions(alpha = 0.05)
res  <- runSimsSingleCohort(bio, dopt, aopt, verbose = FALSE)
rowMeans(res$marginal_power)  # mean power at each N
```

---

sampleTimesFromDist *Sample Time Points from an Input-Time Distribution*

Description

Draws sampling times from a circular kernel density estimate of observed input times, or from the saved aggregate distribution in a bundled pilot.

Usage

```
sampleTimesFromDist(n, cts)
```

Arguments

n	Integer. Number of sampling times to draw.
cts	Numeric vector of observed times in hours, or the time_distribution object stored in a bundled GTEx pilot.

Value

Numeric vector of length n with sampled times in hours.

---

scp_load_pilot *Load a Bundled SCP Pilot Dataset*

Description

The returned object is a CircadianBioOptions pilot from the bundled database, produced by the standard cosinor pipeline and ready for power simulation. It has the same form as a pilot you calibrate from your own data, so it can be passed straight to the single-cohort and differential power runners.

Usage

```
scp_load_pilot(
  species,
  dataset,
  tissue,
  condition = NULL,
  alpha_pilot = 0.01,
  adjust = c("none", "BH"),
  K = 1,
  paired_sigma = TRUE,
  time_basis = NULL
)
```

Arguments

species	Character, length 1. One of "human", "baboon", "other_primate", "mouse", "rat".
dataset	Character, length 1. Manifest dataset field (typically GEO accession or short label, e.g. "GTEx", "GSE160521").
tissue	Character, length 1. Tissue or region label as recorded in the manifest (e.g. "Liver", "NAc").
condition	Character, length 1, optional. Condition arm (e.g. "Control", "All"). If NULL and the (species, dataset, tissue) triple identifies more than one row, the function errors and prints the available conditions.
alpha_pilot	Numeric in (0, cap]. Rhythmicity threshold used to re-derive prop_rhythmic and the amplitude/phase/sigma distributions from the pilot's stored per-gene rhythm table. Default 0.01 matches the build-time threshold. Only takes effect for pilots rebuilt with a rhythm_fit table; older pilots ignore it (with a warning) and return their frozen build-time distributions.
adjust	Multiple-testing adjustment applied to the per-gene p-values before thresholding at alpha_pilot: "none" (raw cosinor p, default) or "BH" (Benjamini-Hochberg q-value).
K	Harmonic order of the requested pilot: 1 (single-harmonic cosinor, default) or 2 (two-harmonic). For K = 2 the two-harmonic variant of the pilot (<file>_2H.rds) is loaded; an error is raised if no such variant has been built for the pilot.
paired_sigma	Logical (default TRUE). Keep the per-gene (A, sigma) pairing so the simulator draws amplitude and noise from the same pilot gene, preserving the realistic narrow $r = A/\sigma$ distribution (the marginal-power / Fig 1-2 Panel A convention). FALSE decouples sigma from amplitude, giving the wide r used by the effect-size-stratified panels (Fig 1-2 Panels B/C).
time_basis	GTEx input time. NULL (default) or "TOD" selects season-adjusted time of death. Other datasets require NULL.

Value

A CircadianBioOptions object with empirical distributions of mesor, log-sigma, amplitude, phase, and proportion rhythmic, ready for downstream simulation.

See Also

estCircadianParam to build the same object from your own pilot data.

Examples

```
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All")
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All",
                      alpha_pilot = 0.05)
bio <- scp_load_pilot("human", "GTEx", "Adrenal", "All",
                      alpha_pilot = 0.1, adjust = "BH")
```

---

scp_pilot_database *SCP Pilot Dataset Database*

Description

The SCP package bundles a curated database of public circadian transcriptomic pilot datasets that users can plug into `runSimsSingleCohort()` or `runDifferentialPower()` as the parameter source for power simulation. Each bundled pilot is a pre-fit `CircadianBioOptions` object stored under `inst/extdata/pilots/<species>/`, alongside a CSV manifest that documents provenance (accession, citation, license), study design (active vs passive), and condition arm.

Three user-facing functions are exposed:

`scp_pilots` Return the manifest as a data.frame, optionally filtered by species and ingestion status.

`scp_load_pilot` Load and return one standardized pilot object given species, dataset, tissue, and condition.

`scp_pilot_search` Convenience search: filter the manifest by free-text query, species, tissue regex, and in-vivo vs cell-line sample type.

---

scp_pilot_search *Search the SCP Pilot Database*

Description

Convenience filter over the pilot manifest. Combines free-text substring matching against dataset, tissue, citation, and accession fields with optional species and tissue-regex filters and an in-vivo vs cell-line restriction. Useful when a user wants to pick a pilot whose design (tissue, sample size, signal strength) matches their planned study.

Usage

```
scp_pilot_search(
  query = NULL,
  species = NULL,
  tissue_pattern = NULL,
  sample_type = "in_vivo",
  time_basis = NULL
)
```

Arguments

query	Character, optional. Case-insensitive substring matched against dataset, tissue, accession, citation. NULL matches everything.
species	Character, optional. Restrict to these species.
tissue_pattern	Character, optional. Regular expression matched against tissue (case-insensitive).
sample_type	Character. Default "in_vivo". Use NULL to include cell-line pilots as well.
time_basis	GTEx input time. NULL (default), "TOD", and "all" search the current v10 pilots with season-adjusted time of death. Other datasets retain native time.

Value

A data.frame: rows of the manifest passing every active filter.

Examples

```
scp_pilot_search("liver")
scp_pilot_search(species = "mouse", tissue_pattern = "brain|cortex")
scp_pilot_search(sample_type = NULL)   # include cell lines
```

---

scp_pilots *List Bundled SCP Pilot Datasets*

Description

Reads the bundled pilot manifest (inst/extdata/pilots/manifest.csv) and returns it as a data.frame. Optionally filters by species and ingestion status so a user can quickly enumerate the pilots they actually have available.

Usage

```
scp_pilots(species = NULL, status = "ingested", time_basis = NULL)
```

Arguments

species	Optional character. One or more of "human", "baboon", "other_primate", "mouse", "rat". NULL (default) returns all species.
status	Character. Filter on the status column. Defaults to "ingested" (datasets actually present on disk). Use NULL or "all" to return every row, including "pending", "verify_failed", "controlled", and "ingest_failed" entries. GTEx rows always use the current v10 TOD pilots.
time_basis	GTEx input time. NULL (default), "TOD", and "all" list the current v10 pilots with season-adjusted time of death (TOD). Other datasets retain native time.

Value

A data.frame with one row per (dataset, tissue, condition, time basis) pilot. Columns: species, dataset, tissue, condition, n, ngenes, design, assay, sample_type, accession, citation, license, file, status.

Examples

```
scp_pilots()
scp_pilots(species = "human")
scp_pilots(status = "all")
```

SeqModelSel *Sequential Model Selection for Joint Rhythmicity*

Description

Sequential Model Selection for Joint Rhythmicity

Usage

```
SeqModelSel(
  action = c(1, 2),
  pv = c(0.01, 0.02),
  alpha = 0.05,
  method = "Fisher_FS"
)
```

Arguments

action	Vector indicating which group to test first
pv	P-values for the tests
alpha	Significance threshold
method	Selection method: "Fisher_BS", "Fisher_FS", "Sidak_BS", "Sidak_FS", "Nominal_BS", "Nominal_FS", "VDA", "AWFisher"

Value

Joint rhythmicity category

---

simCircadianDiffFMM *Simulate Two-Group Differential Circadian Data under FMM Waveform*

Description

Extends simCircadianDiff by replacing the cosinor generative model with the FMM (Frequency Modulated Mobius) waveform. Gene-type assignment (DR/DP/DM/null) and parameter draws are identical to simCircadianDiff; only expression generation uses FMM.

Uses a precomputed unit-amplitude FMM template (one per omega value) and per-gene interpolation, avoiding the per-gene FMM::generateFMM() call and giving a ~1000x speed-up over naive implementation.

Beta-invariance: The orientation parameter beta shifts the peak location but does not alter amplitude. DCP differential-amplitude power (DR, DP) is therefore invariant to beta. Only phase estimation (DP peak-time accuracy) is affected by beta.

omega = 1 identity: When omega = 1 the FMM waveform reduces exactly to the cosinor model; this function returns the simCircadianDiff result unmodified for efficiency.

Usage

```
simCircadianDiffFMM(..., omega = 1, beta = pi)
```

Arguments

`...`	All other arguments passed directly to `simCircadianDiff`.
`omega`	Numeric in [0, 1]. FMM waveform shape: `1` = pure cosinor (default); `0` = flat (arrhythmic); values in $(0, 1)$ give increasingly peaked non-sinusoidal waveforms.
`beta`	Numeric. FMM orientation parameter (default π gives a symmetric peak at `alpha`). Does not affect amplitude-based power (beta-invariance).

Value

Same list structure as `simCircadianDiff`.

References

Rueda C, Rodriguez-Collado A, Peddada SD (2019). "A single notable oscillation in humans and mammals with strong implications in chronobiology." Sci Rep 9, 17982. doi:10.1038/s41598019-545691

See Also

`simCircadianDiff`, `simCircadianFMM`, `plotFMMDifferential`

---

simCircadianSingleCohort2H

Simulate Two-Harmonic Single-Cohort Data

Description

Two-harmonic analog of the single-harmonic single-cohort simulator. Generates expression from

$$y_{gi} = \mu_g + A_{g,1}\cos(\omega_0(t_i - \phi_{g,1})) + A_{g,2}\cos(2\omega_0(t_i - \phi_{g,2})) + \epsilon_{gi}$$

using the joint pilot tuple $(A_1, \phi_1, A_2, \phi_2, \sigma)$ stored on `bio.opts` by `estCircadianParam2H`. The same index is drawn across all five vectors so the pilot's cross-parameter dependence is preserved.

`r_values` is reported on the 1st-harmonic scale $r_g = A_{g,1}/\sigma_g$ to match the existing r-stratification machinery and the manuscript's r_g notation.

Usage

```
simCircadianSingleCohort2H(bio.opts, cts, seed = NULL)
```

Arguments

`bio.opts`	A `CircadianBioOptions` produced by `estCircadianParam2H` (must carry `paired_2h = TRUE`, `amplitude2`, `phase2`).
`cts`	Numeric vector of sample collection times in hours; length determines N.
`seed`	Optional integer random seed.

Value

Named list:

`expr` Gene expression matrix (`ngenes` x N).

`is_rhythmic` Logical vector.

`r_values` Per-gene 1st-harmonic SNR $A_{g,1}/\sigma_g$.

`A1`, `phi1`, `A2`, `phi2` Per-gene truth vectors (length `ngenes`; zero for non-rhythmic genes).

See Also

`estCircadianParam2H` for the matching pilot fit.

Examples

```
# Generate one two-harmonic dataset from a 2H pilot fit.
praw <- readRDS(system.file("extdata", "example_pilot_raw.rds",
                            package = "SCP"))
expr <- praw$adrenal$expr[1:300, ]
bio2h <- estCircadianParam2H(expr, praw$adrenal$times,
                             top_k = 100, verbose = FALSE)
bio2h$ngenes <- 300L
cts <- rep(seq(0, 22, by = 2), 2)
sim <- simCircadianSingleCohort2H(bio2h, cts, seed = 1)
dim(sim$expr)
```

StopRule *Stop Rule for Sequential Testing*

Description

Stop Rule for Sequential Testing

Usage

```
StopRule(action, pv, alpha, method)
```

Arguments

`action`	Vector indicating which group to test first
`pv`	P-values
`alpha`	Significance threshold
`method`	"FS" (forward stop) or "BS" (basic stop)

Value

List with type and one